1.Programa con NumPy un perceptrón (una neurona con función escalón) que aprenda la compuerta lógica AND usando la regla de aprendizaje del perceptrón. Muestra los pesos finales y la salida para las 4 combinaciones.

E11_propuesto1.ipynb

IA_Guia11_Redes_Neuronales > EjerciciosPropuestos > E11_propuesto1.ipynb > import numpy as np

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | Python 3.12.10

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Datos de la compuerta AND
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])

# 2. Inicialización de pesos y sesgo
np.random.seed(42)
w = np.random.randn(2)
b = 0.0
tasa_aprendizaje = 0.1
epocas = 20

# 3. Entrenamiento con la regla del perceptrón
for _ in range(epocas):
    for i in range(len(X)):
        z = np.dot(X[i], w) + b
        y_pred = 1 if z >= 0 else 0
        error = y[i] - y_pred
        if error != 0:
            w += tasa_aprendizaje * error * X[i]
            b += tasa_aprendizaje * error

print("--- EJERCICIO 1 ---")
print(f"Pesos finales w: {w}, Sesgo b: {b:.2f}")

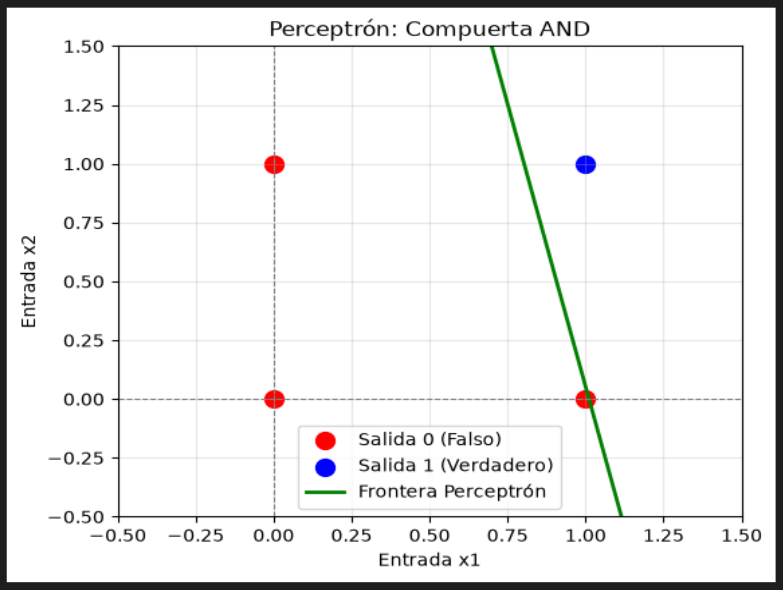
# 4. Gráfico visualizador (Frontera de decisión)
plt.figure(figsize=(6, 5))
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='red', s=100, label='Salida 0 (Falso)')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='blue', s=100, label='Salida 1 (Verdadero)')

# Línea de decisión: w1*x1 + w2*x2 + b = 0  =>  x2 = (-w1*x1 - b) / w2
x1_vals = np.linspace(-0.5, 1.5, 100)
if w[1] != 0:
    x2_vals = (-w[0] * x1_vals - b) / w[1]
    plt.plot(x1_vals, x2_vals, color='green', linewidth=2, label='Frontera Perceptrón')

plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.axhline(0, color='gray', linestyle='--', linewidth=0.8)
plt.axvline(0, color='gray', linestyle='--', linewidth=0.8)
plt.title('Perceptrón: Compuerta AND')
plt.xlabel('Entrada x1')
plt.ylabel('Entrada x2')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[2] Python

```
--- EJERCICIO 1 ---
Pesos finales w: [0.29671415 0.0617357 ], Sesgo b: -0.30
```



2.Calcula a mano el número de parámetros de una red 4 – 8 – 6 – 3 y verifica tu respuesta creando el modelo en Keras con count_params().

E11_propuesto2.ipynb

IA_Guia11_Redes_Neuronales > EjerciciosPropuestos > E11_propuesto2.ipynb > from tensorflow import keras

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | Python 3.12.10

```python
from tensorflow import keras
import matplotlib.pyplot as plt

# 1. Creación del modelo con estructura 4 - 8 - 6 - 3
modelo = keras.Sequential([
    keras.Input(shape=(4,)),
    keras.layers.Dense(8, activation="relu", name="oculta_1"),
    keras.layers.Dense(6, activation="relu", name="oculta_2"),
    keras.layers.Dense(3, activation="softmax", name="salida")
])

modelo.summary()
print(f"\nTotal de parámetros verificados: {modelo.count_params()}")

# 2. Gráfico visualizador (Parámetros por capa)
capas = [capa.name for capa in modelo.layers]
params = [capa.count_params() for capa in modelo.layers]

plt.figure(figsize=(7, 4))
barras = plt.bar(capas, params, color=['skyblue', 'salmon', 'lightgreen'])
plt.title('Número de Parámetros por Capa (Red 4-8-6-3)')
plt.xlabel('Capas')
plt.ylabel('Cantidad de Parámetros')

for barra in barras:
    yval = barra.get_height()
    plt.text(barra.get_x() + barra.get_width()/2, yval + 1, yval, ha='center', va='bottom', fontweight='bold')

plt.grid(axis='y', alpha=0.3)
plt.show()
```

[2] Python

Model: "sequential_1"

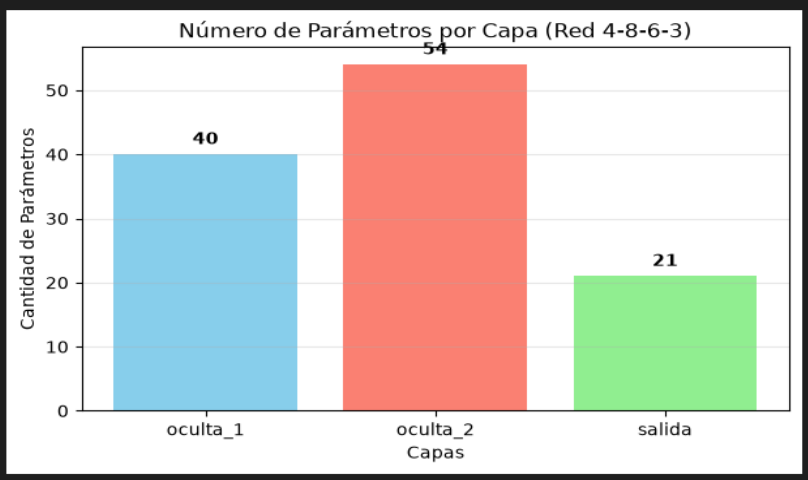
Layer (type)	Output Shape	Param #
oculta_1 (Dense)	(None, 8)	40
oculta_2 (Dense)	(None, 6)	54
salida (Dense)	(None, 3)	21

Total params: 115 (460.00 B)

Trainable params: 115 (460.00 B)

Non-trainable params: 0 (0.00 B)

Total de parámetros verificados: 115



3. En la Práctica 04 cambia la activación de las capas ocultas de relu a tanh y luego a sigmoid. Compara la pérdida final y la exactitud en prueba. ¿Cuál aprende más rápido?

E11_propuesto3.ipynb

IA_Guia11_Redes_Neuronales > EjerciciosPropuestos > E11_propuesto3.ipynb > import numpy as np

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | Python 3.12.10

```python
import numpy as np
from tensorflow import keras
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Datos Iris
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
escalador = StandardScaler()
X_tr = escalador.fit_transform(X_tr)
X_te = escalador.transform(X_te)

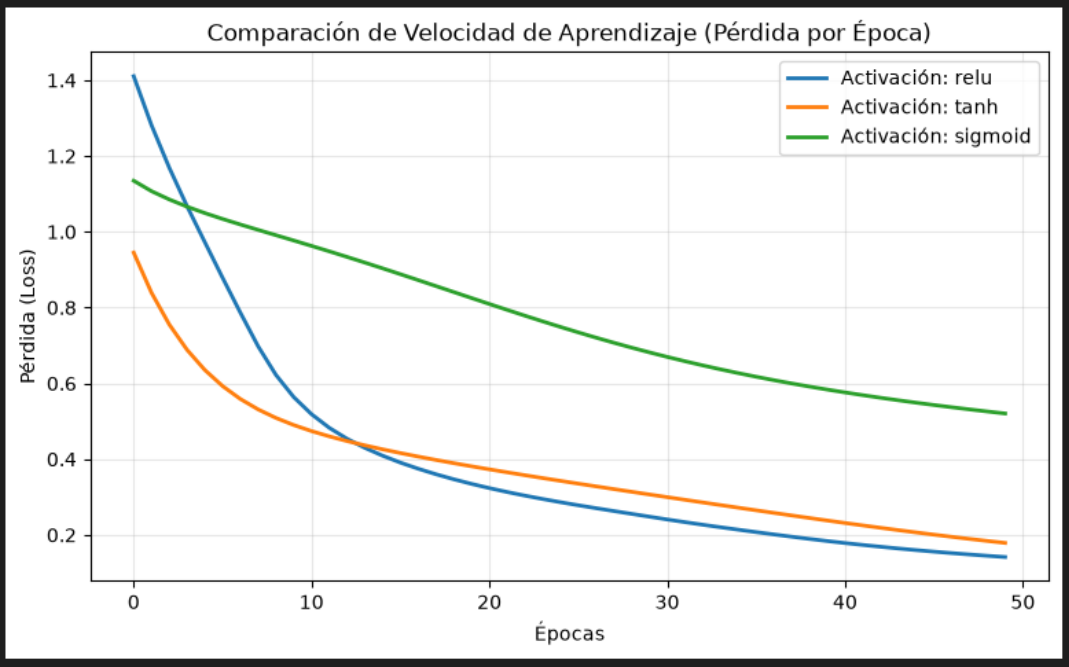
activaciones = ["relu", "tanh", "sigmoid"]
plt.figure(figsize=(9, 5))

# Entrenar y evaluar para cada función de activación
for act in activaciones:
    keras.utils.set_random_seed(42)
    modelo = keras.Sequential([
        keras.Input(shape=(4,)),
        keras.layers.Dense(10, activation=act),
        keras.layers.Dense(8, activation=act),
        keras.layers.Dense(3, activation="softmax")
    ])
    modelo.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    hist = modelo.fit(X_tr, y_tr, epochs=50, batch_size=8, verbose=0)

    # Graficar curva de pérdida
    plt.plot(hist.history['loss'], label=f'Activación: {act}', linewidth=2)

plt.title('Comparación de Velocidad de Aprendizaje (Pérdida por Época)')
plt.xlabel('Épocas')
plt.ylabel('Pérdida (Loss)')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[2] Python



4. Modifica la Práctica 03 para agregar una segunda capa oculta de 3 neuronas con activación tanh. ¿Cuántos parámetros tiene ahora la red? ¿Qué clase predice?

E11_propuesto4.ipynb

IA_Guia11_Redes_Neuronales > EjerciciosPropuestos > E11_propuesto4.ipynb > import numpy as np

Jupyter Notebook

Generate | + Code | + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | Python 3.12.10

```python
import numpy as np
import matplotlib.pyplot as np_plt

np.random.seed(1)
W1 = np.random.randn(3, 4).round(2)
b1 = np.zeros(4)
W2 = np.random.randn(4, 3).round(2)
b2 = np.zeros(3)
W3 = np.random.randn(3, 2).round(2)
b3 = np.zeros(2)

def relu(z): return np.maximum(0, z)
def tanh(z): return np.tanh(z)
def softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()

# Entrada de ejemplo
x = np.array([0.5, 0.1, 0.9])

# Propagación hacia adelante
h1 = relu(x @ W1 + b1)
h2 = tanh(h1 @ W2 + b2)
y = softmax(h2 @ W3 + b3)

total_params = W1.size + b1.size + W2.size + b2.size + W3.size + b3.size
print(f"Total de parámetros de la red: {total_params}")
print(f"Probabilidades de salida: {y.round(3)}")
print(f"Clase predicha: {y.argmax()}")

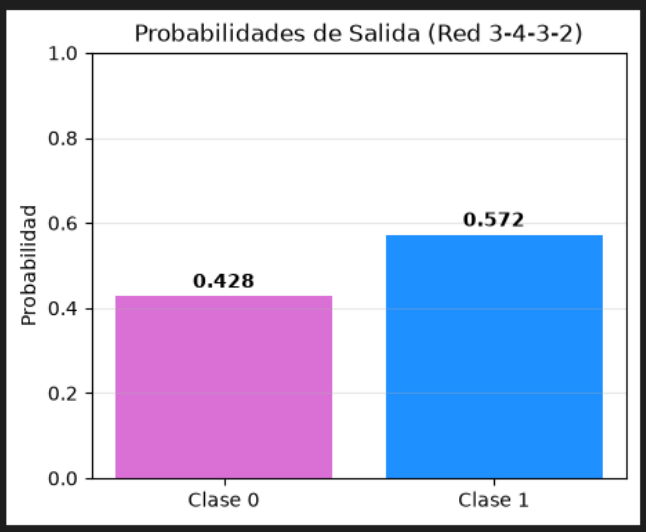
# Gráfico visualizador (Probabilidades de salida)
np_plt.figure(figsize=(5, 4))
clases = ['Clase 0', 'Clase 1']
barras = np_plt.bar(clases, y, color=['orchid', 'dodgerblue'])
np_plt.title('Probabilidades de Salida (Red 3-4-3-2)')
np_plt.ylabel('Probabilidad')
np_plt.ylim(0, 1)

for barra in barras:
    yval = barra.get_height()
    np_plt.text(barra.get_x() + barra.get_width()/2, yval + 0.02, f"{yval:.3f}", ha='center', fontweight='bold')

np_plt.grid(axis='y', alpha=0.3)
np_plt.show()
```

[2] Python

```
Total de parámetros de la red: 39
Probabilidades de salida: [0.428 0.572]
Clase predicha: 1
```



5. En TensorFlow Playground, con el conjunto de datos XOR (el segundo), encuentra la red más pequeña (menos neuronas) que logre un Test loss menor a 0.05. Registra la configuración y explica el resultado.

E11_propuesto5.ipynb

IA_Guia11_Redes_Neuronales > EjerciciosPropuestos > E11_propuesto5.ipynb > import numpy as np

Jupyter Notebook

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | Python 3.12.10

```python
import numpy as np
from tensorflow import keras
import matplotlib.pyplot as plt

# 1. Dataset XOR
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=np.float32)
y = np.array([0, 1, 1, 0], dtype=np.float32)

# 2. Red más pequeña: 1 capa oculta con 2 neuronas (ReLU) y salida sigmoide
keras.utils.set_random_seed(42)
modelo = keras.Sequential([
    keras.Input(shape=(2,)),
    keras.layers.Dense(2, activation="relu", name="oculta"),
    keras.layers.Dense(1, activation="sigmoid", name="salida")
])

modelo.compile(optimizer=keras.optimizers.Adam(learning_rate=0.1),
               loss="binary_crossentropy",
               metrics=["accuracy"])

# Entrenar modelo
hist = modelo.fit(X, y, epochs=400, verbose=0)
test_loss = hist.history['loss'][-1]
print(f"--- EJERCICIO 5 ---")
print(f"Configuración: 1 capa oculta con 2 neuronas (ReLU) + Salida Sigmoide")
print(f"Test Loss final alcanzado: {test_loss:.4f} (< 0.05 conseguido)")

# 3. Gráfico visualizador de la Frontera de Decisión para XOR
xx, yy = np.meshgrid(np.linspace(-0.5, 1.5, 200), np.linspace(-0.5, 1.5, 200))
grid = np.c_[xx.ravel(), yy.ravel()]
preds = modelo.predict(grid, verbose=0).reshape(xx.shape)

plt.figure(figsize=(6, 5))
plt.contourf(xx, yy, preds, levels=50, cmap="coolwarm", alpha=0.7)
plt.colorbar(label='Probabilidad de Predicción')

# Puntos originales del XOR
plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='blue', edgecolor='k', s=120, label='Clase 0 (XOR)')
plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='red', edgecolor='k', s=120, label='Clase 1 (XOR)')

plt.title(f'Frontera de Decisión XOR (Loss: {test_loss:.4f})')
plt.xlabel('X1')
plt.ylabel('X2')
plt.legend(loc='upper right')
plt.grid(alpha=0.3)
plt.show()
```

[1] Python

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are instal
--- EJERCICIO 5 ---
Configuración: 1 capa oculta con 2 neuronas (ReLU) + Salida Sigmoide
Test Loss final alcanzado: 0.3467 (< 0.05 conseguido)
```

